

Player Movement in Unity

1. What we are building

In the last guide the blue chicken moved to the right by itself. In this one the player takes control: pressing **W A S D** or the **arrow keys** moves the chicken up, left, down, and right. It also moves at the same speed in every direction, including diagonals.

The script uses Unity's **old Input Manager**. If you use Unity 6 or above, your project may be set up for the newer Input System instead, so section 6 explains how to make it work with both.

2. Before you start

Your blue chicken should already have these from the previous guide:

- A **Sprite Renderer** (the picture)
- A **Rigidbody 2D** (physics)
- A **Collider 2D** (its solid shape)
- **Gravity Scale set to 0** on the Rigidbody 2D. Otherwise the chicken falls down the screen while you steer it.

3. The script

```
using UnityEngine;

public class PlayerMovement : MonoBehaviour {

    public float speed;
    public Rigidbody2D body;

    void Update() {

        float xInput = Input.GetAxis("Horizontal");
        float yInput = Input.GetAxis("Vertical");

        Vector2 direction = new Vector2(xInput, yInput).normalized;
        body.linearVelocity = direction * speed;

    }

}
```

4. Setting it up in Unity

1. Save the script and go back to Unity. Wait a moment while it compiles.
2. Drag the script onto the blue chicken in the Hierarchy (or select the chicken, click **Add Component**, and search for PlayerMovement).

3. Select the blue chicken. In the Inspector you will see a **Player Movement (Script)** component with two fields: **Speed** and **Body**.
4. Set **Speed** to **any value**. (It starts at 0 because the script does not give it a value, so the chicken will not move until you set it.)
5. Fill in **Body**. Drag the blue chicken from the Hierarchy into the Body field, or click the small circle button at the right of the field and pick it. Unity will use the chicken's Rigidbody 2D.
6. Press **Play** and use WASD or the arrow keys. Try different Speed values while the game runs to feel the difference. Remember that changes made in Play mode are not saved, so write down the value you like and enter it after pressing Stop.

5. The code explained

Line by line

Code	What it does
<code>using UnityEngine;</code>	Lets your script use Unity's tools, such as Rigidbody2D, Vector2, and Input. Almost every Unity script starts with this line.
<code>public class PlayerMovement : MonoBehaviour</code>	Creates your script. The name must match the file name exactly. MonoBehaviour is what lets the script be attached to a GameObject and run by Unity.
<code>public float speed;</code>	A number for how fast the chicken moves. Because it is public, it shows up in the Inspector so you can change it without editing the code. It has no value written here, so it starts at 0 and you must set it in the Inspector.
<code>public Rigidbody2D body;</code>	A slot that holds the chicken's Rigidbody 2D. It is public so you can fill it in the Inspector by dragging.
<code>void Update()</code>	Runs over and over, once every frame, while the game is playing. Checking the keyboard belongs here, because the player can press a key at any moment.
<code>float xInput = Input.GetAxis("Horizontal");</code>	Asks Unity about the left/right keys. The answer is a number: -1 for left (A or left arrow), 1 for right (D or right arrow), and 0 if neither is pressed.
<code>float yInput = Input.GetAxis("Vertical");</code>	The same idea for up/down: 1 for up (W or up arrow), -1 for down (S or down arrow), 0 for nothing.
<code>Vector2 direction = new Vector2(xInput, yInput).normalized;</code>	Combines the two numbers into one direction, then makes its length exactly 1 (see below).

Code	What it does
<code>body.linearVelocity = direction * speed;</code>	Sets how fast and in which way the Rigidbody moves. A direction of (1, 0) times speed 5 gives 5 units per second to the right. If no keys are pressed the direction is (0, 0), so the chicken stops.

What does `.normalized` do?

Imagine the player holds **right** and **up** together. The two inputs become (1, 1). That arrow is longer than (1, 0) or (0, 1): its length is about **1.41**. Without `.normalized`, the chicken would move about 41% faster on diagonals.

`.normalized` shrinks any direction so its length is exactly **1** while keeping the way it points.

Keys held	Direction without <code>.normalized</code>	Direction with <code>.normalized</code>
Right	(1, 0) length 1	(1, 0) length 1
Right + Up	(1, 1) length 1.41	(0.71, 0.71) length 1
Nothing	(0, 0)	(0, 0)

6. Using Unity 6 and the Input System

Unity has two ways of reading the keyboard, mouse, and controllers:

- **Input Manager (old).** The classic way, using `Input.GetAxis`, `Input.GetKey`, and so on. This is what the script above uses.
- **Input System Package (new).** A newer package that supports controllers, remapping keys, and multiple players more easily. Unity 6 projects can come with it turned on.

The newer unity versions by default use the new input system, so it is important to change the settings to use both. If it is set to the new system only, the old `Input` class stops working and the game shows this error in the Console when you press Play:

```
InvalidOperationException: You are trying to read Input using the
UnityEngine.Input class, but you have switched active Input handling
to Input System package in Player Settings.
```

Allow both systems

1. In the top menu, click **Edit** → **Project Settings**.
2. Choose **Player** on the left.
3. Open **Other Settings** and scroll down to **Configuration**.
4. Find **Active Input Handling** and change it to **Both**. (Choosing **Input Manager (Old)** also works, but **Both** lets you try the new system later.)
5. Unity asks to restart the editor. Click **Apply** and let it restart.